Chris Toomey, Jenice An, and Tongyu Guo
ADAN 8888 - Applied Analytics Project - Spring 2026
Module 9 Assignment - Select the Winning Model

HelpHerInvest: A machine learning-based investment recommendation system that aligns user interests and risk preferences with benchmark stock evaluation.

Nine Modeling Approaches

For our first modeling approach, we selected logistic regression since our objective is to classify stocks based on their forward excess return relative to the S&P 500, which makes this a supervised classification problem rather than a continuous forecasting task. Logistic regression is well suited because it provides us a structured way to examine how financial indicators relate to future performance outcomes. As a first step, it allows us to test whether our features have predictive value. By starting with a simpler model, we are able to establish a baseline level of performance before exploring more complex techniques.

For our second modeling approach, we've selected to use a light gradient-boosting machine (LGBM) model to apply a decision tree based classifier. A LGBM model is similar to random forests but it builds the trees sequentially and attempts to correct and learn from errors for each new tree. It is particularly useful when dealing with larger datasets, which is useful for the technical dataset we are working with. The LGBM model grows the trees vertically at the leaf-wise, which is unlike other decision tree models (i.e. XGBoost) that grow horizontally at the level-wise. This allows the model to prioritize splitting the leaf that produces the max reduction in the loss function, which can result in more accurate trees with fewer overall splits.

Lastly, we implemented neural network models using TensorFlow for the third and final modeling approach. We felt utilizing variations of neural network models would be helpful in capturing the non-linear relationships between our independent variables and the dependent variable. The goal is to have the neural network model smooth the noisy relationships between the technical dataset variables we are working with.

1. **L1 (Lasso) Logistic Regression**
2. **Elastic Net Logistic Regression**
3. **Weighted Logistic Regression**

4. **LGBM Model 1:**

```
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=31
)
```

5. **LGBM Model 2 - balanced weights:**

```
model2 = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=12,
    num_leaves=31,
    class_weight="balanced",
)
```

6. **LGBM Model 3 - lightweight:**

```
model3 = lgb.LGBMClassifier(
    n_estimators=500,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=20
)
```

7. **Baseline Neural Network Model:**

```python
def build_mlp_baseline(input_dim):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="auc")
        ]
    )
    return model
```

8. **Regularized Neural Network Model:**

```python
def build_mlp_regularized(input_dim):
    l2 = regularizers.l2(1e-4)

    inputs = keras.Input(shape=(input_dim,))
    x = layers.Dense(256, kernel_regularizer=l2)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.35)(x)

    x = layers.Dense(128, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.30)(x)

    x = layers.Dense(64, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.20)(x)

    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=5e-4),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="auc")
        ]
    )
    return model
```

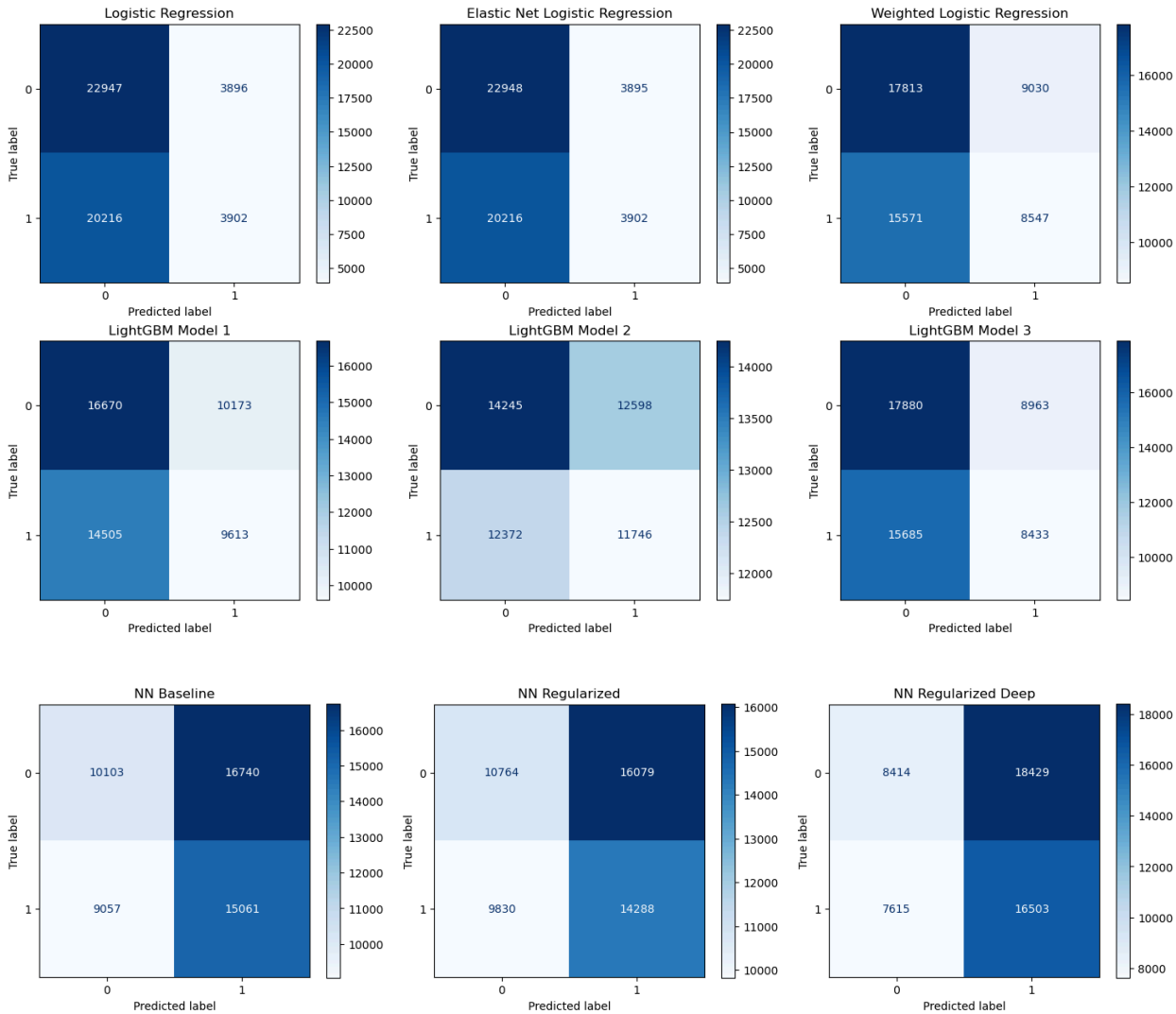
9. **Regularized Deep Neural Network Model:**

```
def build_mlp_regularized_deep(input_dim):
    l2 = regularizers.l2(1e-4)

    inputs = keras.Input(shape=(input_dim,))
    x = layers.Dense(256, kernel_regularizer=l2)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.35)(x)

    x = layers.Dense(128, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.30)(x)

    x = layers.Dense(64, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.10)(x)

    x = layers.Dense(32, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.05)(x)

    x = layers.Dense(16, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=5e-4),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="auc")
        ]
```

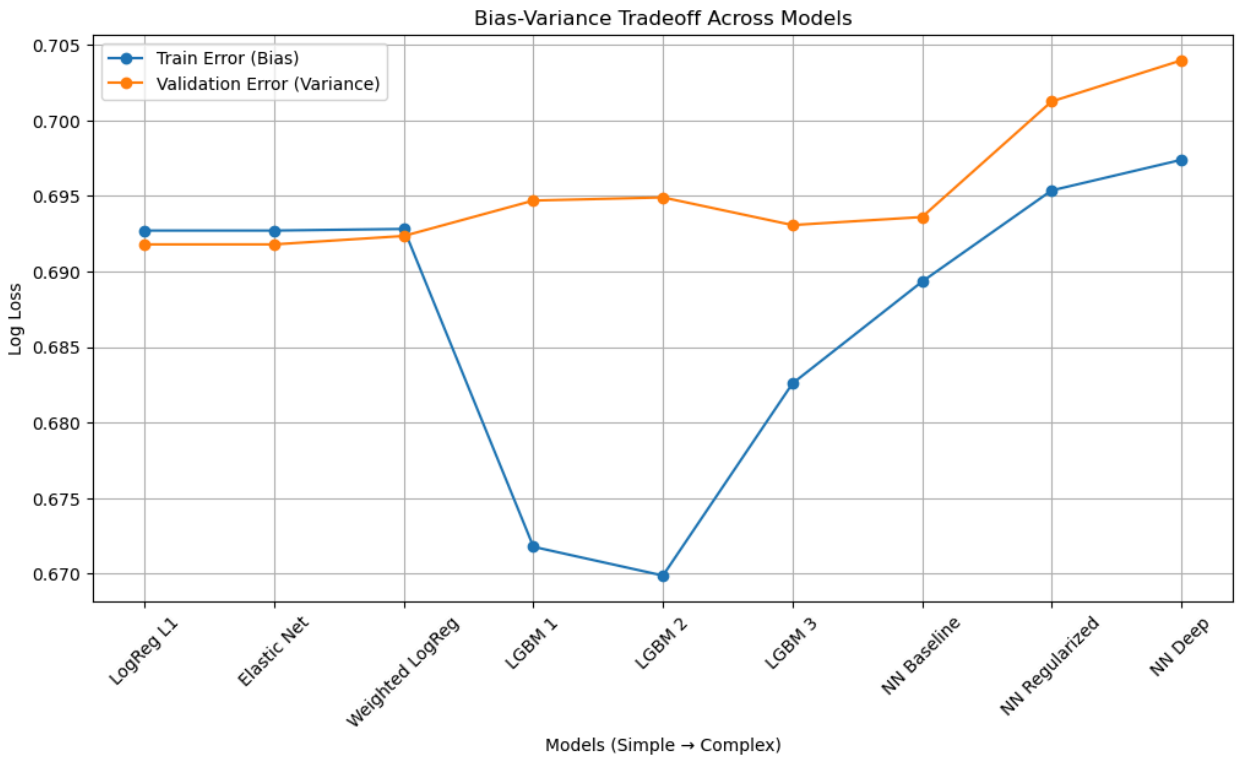
Validation Error For All 9 Models

Models	F1 Score
LightGBM Model 2	0.5088
LightGBM Model 1	0.5063
Weighted Logistic Regression	0.5008
LightGBM Model 3	0.4991
Regularized NN Model	0.4891
Baseline NN Model	0.4890
Regularized Deep NN Model	0.4757
Elastic Net Logistic Regression	0.4501
Logistic Regression	0.4500

We decided to focus on the F1 score because it provides you with a blended metric, accounting for both the precision and recall results. Overall, the balanced weight Light Gradient Boost Model (LightGBM Model 2) had the best F1 score out of all the models we ran. This was a little surprising to our team because we had anticipated one of the neural network models to outperform the LightGBM models. Had we done some extra hyperparameter tuning and feature engineering, we may have seen better results. Ultimately, I think the shortcomings of our models' predictions stem from the dataset having high levels of multicollinearity since they often are calculated in similar methods.


Logistic Regression
22947
3896
20216
3902
Elastic Net Logistic Regression
22948
3895
20216
3902
Weighted Logistic Regression
17813
9030
15571
8547
LightGBM Model 1
16670
10173
14505
9613
LightGBM Model 2
14245
12598
12372
11746
LightGBM Model 3
17880
8963
15685
8433
NN Baseline
10103
16740
9057
15061
NN Regularized
10764
16079
9830
14288
NN Regularized Deep
8414
18429
7615
16503
True label
Predicted label

Bias-Variance Tradeoff



It appears the logistic regression (simpler models) display a high validation and training error, which indicates bias because they fail to capture complex relationships in the data. Despite being the most complex out of the collection, the neural network models do not improve the validation performance - suggesting that there is a low signal in the dataset. The LightGBM models have a slightly lower training error compared to the validation error, which is a positive sign. Overall, it seems like the LightGBM models reduce the bias and achieve the best overall performance.

Winning Model Performance Metrics

```
LightGBM Model 2 - Test Set:
Classification Report:

              precision    recall  f1-score   support

           0       0.55      0.53      0.54     25892
           1       0.48      0.50      0.49     22601

    accuracy                           0.52     48493
   macro avg       0.52      0.52      0.52     48493
weighted avg       0.52      0.52      0.52     48493

F1: 0.5155
ROC-AUC: 0.5215
```

Our strongest performing model based on the F1 score was the LightGBM Model 2, using the balanced class weight. Once again, the difference between the validation and testing sets was relatively similar, but the test results were slightly better. For example, the validation dataset had an F1 score of 0.5088, whereas the test set had a F1 score of 0.5200. The closeness in the two scores indicates that the model is not overfitting and can generalize the data fairly well, even with the dataset constraints.

Out of curiosity, we ran each model against the test dataset to see how their performance compared against the validation dataset. The baseline neural network model slightly outperformed the LightGBM Model 2 that we had selected when reviewing the F1 score. The LightGBM Model 2 did have the second best results, which was reassuring.

```
Neural Network Baseline:

NN Baseline
Accuracy: 0.5418
F1: 0.5346

Classification Report
              precision    recall  f1-score   support

           0     0.5642    0.6233    0.5923     25892
           1     0.5096    0.4483    0.4770     22601

    accuracy                         0.5418     48493
   macro avg     0.5369    0.5358    0.5346     48493
weighted avg     0.5387    0.5418    0.5385     48493
```